

Reward Accounting Drift, Security Review

Target: `RewardPool`, a MasterChef-style single staking pool **Type:** Demonstration review on intentionally vulnerable code **Method:** Foundry proof-of-concept, vulnerable branch (`master`) and remediated branch (`fixed`)

This report documents one finding. A passing proof-of-concept reproduces the exploit on `master`; the same scenario, run against the remediated contract on `fixed`, shows the attack neutralised.

H-01, Stale accumulator on deposit (High)

A depositor who joins a staking pool right before its reward accumulator is refreshed is credited for rewards that accrued while they held no stake. In the proof-of-concept Alice is the sole staker for 100 seconds and earns all 100 reward tokens of that window; the attacker deposits at the end, harvests immediately, and takes 50 of them. The 50 tokens come straight out of Alice's earned rewards. The attack needs no privilege and no interaction from the victim.

Root cause

The pool uses the standard MasterChef accounting: `accRewardPerShare` accumulates rewards per staked unit, and each user's `rewardDebt` records the accumulator value at their last interaction, so `pending = amount * accRewardPerShare - rewardDebt`. The invariant is that `accRewardPerShare` must be brought current before any stake changes. `deposit` breaks it:

```
function deposit(uint256 amount) external {
    UserInfo storage u = users[msg.sender];
    if (u.amount > 0) { ... }
    stakeToken.transferFrom(msg.sender, address(this), amount);
    u.amount += amount;
    totalStaked += amount;
    u.rewardDebt = (u.amount * accRewardPerShare) / ACC; // stale accRewardPerShare
}
```

No `updatePool()` runs first. When rewards have accrued since the last refresh, `accRewardPerShare` is still the old, lower value. The new depositor's `rewardDebt` is snapshotted against that stale value, so the next refresh credits them for the gap.

Attack

1. Alice deposits 100 at time `t0`. She is the sole staker. `accRewardPerShare` is 0.
2. 100 seconds pass. 100 reward tokens accrue, all owed to Alice. The accumulator has not been refreshed, so it still reads 0 and `lastRewardTime` is still `t0`.
3. Attacker deposits 100 at `t0 + 100`. No `updatePool` runs, so the attacker's `rewardDebt` is set against the stale accumulator value of 0. `totalStaked` becomes 200.
4. Attacker calls `harvest`, which refreshes the accumulator: the 100 accrued tokens are now spread over `totalStaked = 200`, giving `accRewardPerShare = 0.5` per unit. The attacker's pending is 100

* 0.5 - 0 = 50. The attacker harvests 50.

5. Alice harvests $100 * 0.5 - 0 = 50$ instead of the 100 she earned.

Proof of concept

test/RewardPool.poc.t.sol, function test_accountingDrift_stealsAccruedRewards, run on master:

```
rewards accrued while only Alice staked (wei): 100000000000000000000
attacker harvested (wei): 50000000000000000000000
alice harvested (wei): 50000000000000000000000
```

Recommendation

Call `updatePool()` at the start of `deposit`, before the stake or `rewardDebt` is touched:

```
function deposit(uint256 amount) external {
    updatePool();
    UserInfo storage u = users[msg.sender];
    ...
    u.rewardDebt = (u.amount * accRewardPerShare) / ACC; // now current
}
```

Refreshing the accumulator first snapshots `rewardDebt` against the up-to-date `accRewardPerShare`, so a new depositor is credited only from the moment they join. Every stake-changing entry point (`deposit`, `withdraw`, `harvest`) must refresh the accumulator before mutating state. On the `fixed` branch the same proof-of-concept shows the attacker harvesting 0 and Alice keeping the full 100.

Severity

High. The impact is direct theft of accrued rewards from honest stakers. The actor needs no privilege, and the only precondition, depositing while rewards are pending accrual, is the normal state of an active pool.

Scope and disclaimer

`RewardPool` is intentionally vulnerable code written to demonstrate audit methodology end to end. It is not production code and must never be deployed. The finding above is a real vulnerability in this demo contract, reproduced with an executable proof-of-concept, not an invented severity.